```
    "spcontainer-type-key|9",                                          ❷
    1L,                                                                ❸
    SPContainer.CargoAdded("9",SPContainer.Cargo("456","sack",22)),    ❹
    1622299422183L)                                                    ❺
```

❶ Offset
❷ Persistence_id
❸ Sequence_number
❹ Event
❺ Timestamp

10.2.1 Where the rubber meets the road

In most cases, you read the journal to create a side effect, such as creating a table, pushing a topic, or calling an external service. For example, suppose the container company tells you that it needs to know the weight of each container including its load. This data should be passed to the dock operators so they can stack the containers better. To do this, you can read the `eventsByTag` stream to create and update a database table containing the total weight per container. You might imagine it would be better to add a variable to your `SPContainer` for this purpose.

One solution could be to add the `totalWeight` variable to the `SPContainer`. That would work for this particular requirement, but it's hopeless to try to think in advance about all future requirements that might interest the company. You might be able to anticipate some of them, but the company will probably request new insights into the data that you didn't account for.

The better solution is to use the data implicit in the journal. Doing so is better for maintainability. You can read the journal and create the `totalWeight` by aggregating `AddCargo` events by container. And for scalability reasons, it's better to extract the data and store the aggregation in a different location. This way, you decouple reading from writing.

Figure 10.3 shows the components of this approach. Here, an Akka cluster with three nodes reads from the journal and stores the data in another store after aggregation.